24CYS401 – SECURE SOFTWARE ENGINEERING
END SEMESTER LABORATORY EXAMINATION

Duration	3 Hours	**Maximum Marks**	100
Course	Secure Software Engineering	**Examination Type**	End Semester Lab

General Instructions

• Develop a complete secure software model and development workflow for the given problem statement by performing the activities in the specified order.

• Each activity must be derived from and consistent with the outputs of the previous activity. Inconsistencies will affect marks.

• Use suitable laboratory tools such as draw.io/StarUML, Jira, Git/GitHub, Docker, Kubernetes/Minikube, and an available CI/CD tool.

• Submit diagrams, tables, source/configuration files, screenshots/evidence, test results, and final observations.

Integrated Activity Flow

Given Problem Statement → Agile Process Selection → Requirements → Use Cases → Analysis Model → ER/Data Model → DFD/Trust Boundaries → Architecture/Design → UI → Threat Modeling/STRIDE → Information Flow → Vulnerability Analysis → Attack Tree → Product Backlog → Jira/Scrum → Sprint Metrics → Secure Build → Refactoring → Docker → Kubernetes → CI/CD → Security Testing/Fuzzing → Logging/Monitoring → Hardening/Secure Deployment → Final Security Review

Phase 1 – Agile Process and Development Approach [6 Marks]

- Select a suitable Agile approach (e.g., Scrum with XP practices) and justify it.
- Map at least five Agile Manifesto principles to this project.
- Identify two refactoring opportunities and show before/after structure or code.
- State two limitations/risks of Agile for this security-critical system and give mitigations.

Deliverable: Agile approach, Manifesto mapping, refactoring evidence, limitations/mitigations.

Phase 2 – Requirements Engineering [7 Marks]

- Identify stakeholders and user types: Student, Faculty, Administrator and any justified actor.
- Identify functional, non-functional, and security requirements.
- Prioritize important requirements and identify CIA, authentication, authorization and audit requirements.
- Prepare a concise SRS/requirements table.

Deliverable: SRS/requirements table.

Phase 3 – Requirements Analysis and UML [7 Marks]

- Develop a UML Use Case Diagram with actors, major use cases, and suitable include/extend relationships.
- Specify two critical use cases with actor, preconditions, main/alternative/exception flows and postconditions.
- Develop one scenario-based analysis model for 'Attend Examination and Submit Answers'.

Deliverable: Use Case Diagram, two Use Case Specifications and analysis model.

Phase 4 – Data and Information Flow Modeling [7 Marks]

- Develop an ER Diagram with entities, attributes, primary keys, relationships and cardinalities.
- Develop a Level-0/Level-1 DFD.
- Identify external entities, processes, data stores, data flows and trust boundaries.

• Ensure consistency among Use Case, ER and DFD models.

Deliverable: ER Diagram and DFD with Trust Boundaries.

Phase 5 – Software Architecture and Design Engineering [7 Marks]

• Propose and justify a suitable software architecture/style/pattern.
• Refine it into major components/services with responsibilities and interfaces.
• Identify at least four applicable design concepts/patterns.
• Map authentication, examination, evaluation/results and audit/logging to components.

Deliverable: Architecture Diagram, component design and justification.

Phase 6 – User Interface Design [5 Marks]

• Design at least four screens: Login, Student Exam Screen, Faculty Examination Management and Results.
• For each, identify user, goal, navigation, inputs, feedback, error handling and security considerations.
• Apply the Golden Rules: consistency, user control, feedback, error prevention, navigation and visibility.

Deliverable: UI wireframes/screens and rationale.

Phase 7 – Threat Modeling and Security Analysis [10 Marks]

• Identify at least eight assets and classify CIA requirements.
• Apply STRIDE to DFD elements and identify at least 10 threats.
• Prepare a threat table: DFD element, threat, STRIDE category, impact and mitigation.
• Perform Information Flow Analysis for at least three sensitive assets.
• Identify at least six vulnerabilities with affected element, related threat, impact and mitigation.

Deliverable: Asset/CIA, STRIDE, Information Flow and Vulnerability Analysis.

Phase 8 – Attack Tree and Security Architecture Refinement [6 Marks]

• Select a critical attacker goal such as modifying another student's marks or accessing a question paper early.
• Construct an Attack Tree using AND/OR relationships.
• Identify root goal, sub-goals, attack paths and preventive/detective controls.
• Refine the architecture/design to address the highest-risk threats.

Deliverable: Attack Tree and security-refined architecture.

Phase 9 – Product Backlog and Jira/Scrum [7 Marks]

• Convert requirements/use cases into at least 10 user stories using the required 'As a… I want… so that…' format.
• For each story provide Story ID, Epic, Priority and acceptance criteria.
• Create a Jira Scrum project with epics, stories and tasks.
• Divide the backlog into two sprints and define a Sprint Goal for each.

Deliverable: Product Backlog, Jira project and two Sprint Plans.

Phase 10 – Sprint Execution and Scrum Metrics [7 Marks]

• Create the board: TO DO → IN PROGRESS → TESTING → DONE.
• Move tasks through the workflow and provide execution evidence.
• Document a Daily Scrum entry with progress, plan and blockers.
• Prepare the Sprint Burndown Chart.
• Calculate/identify velocity, defects and defects carried over.
• Prepare Sprint Review outcome and Retrospective with at least two improvement actions.

Deliverable: Sprint Board, Daily Scrum, Burndown, Velocity, Defect/Carry-over metrics, Review and Retrospective.

Phase 11 – Secure Development and Build Environment [6 Marks]

- Set up a secure repository and suitable branch/workflow strategy.
- Identify at least five controls: least privilege, secret management, dependency control, code review, protected branches, reproducible builds or artifact integrity.
- Demonstrate that secrets are not hard-coded.
- Apply one automated static/security check and document remediation.

Deliverable: Repository/workflow evidence, secure-build checklist and security-check result.

Phase 12 – Secure Coding and Refactoring [4 Marks]

- Implement a small module such as authentication/authorization, answer submission or result management.
- Identify at least two security weaknesses/code-quality issues in the initial version.
- Refactor/fix them and explain the security/maintainability improvement.
- Demonstrate input validation, authorization, error handling and secure sensitive-data handling.

Deliverable: Source code, before/after evidence and security justification.

Phase 13 – Containerized Development: Docker and Kubernetes [7 Marks]

- Create a Dockerfile and build/run the application container.
- Apply at least four container-security practices such as minimal image, non-root execution, controlled ports, version control, secret handling or removal of unnecessary packages.
- Create Kubernetes Deployment/Service configuration using Kubernetes/Minikube.
- Apply at least two Kubernetes security controls such as resource limits, namespace isolation, non-root security context, restricted exposure or Secret management.

Deliverable: Dockerfile, image/container evidence, Kubernetes manifests and deployment/security evidence.

Phase 14 – CI/CD and Security Testing [7 Marks]

- Create a CI/CD pipeline covering checkout, build, automated test, security/static check and deployment/package stage.
- Write and execute unit tests for at least two functions/modules.
- Perform one integration test and one validation/system end-to-end test.
- Perform a simple fuzzing test on an input boundary and record observations.
- Record at least one defect, severity, fix and retest result.

Deliverable: CI/CD evidence, test results, fuzzing evidence, defect report and retest.

Phase 15 – Logging, Monitoring, Hardening and Secure Deployment [5 Marks]

- Identify security-relevant events that must be logged, such as failed login, privilege changes and result modification.
- Design a basic logging/monitoring strategy with at least five useful metrics/alerts.
- Prepare a target-environment hardening checklist covering access, ports/services, secrets, updates, permissions and least privilege.
- Identify physical and operational security controls and prepare a secure deployment checklist.

Deliverable: Logging/Monitoring plan, hardening checklist, physical/operational controls and deployment evidence.

Phase 16 – Final Security Review [1 Marks]

- Trace one critical requirement through Requirement → Use Case → DFD → Threat → Vulnerability → Attack Tree → User Story → Sprint Task → Implementation → Test → Deployment Control.
- Identify the three highest-risk issues and their controls; state two remaining limitations/future improvements.

Deliverable: Traceability/security review and final observations.

Mark Distribution Summary

Phase	Activity	Marks
1	Agile Process and Development Approach	6
2	Requirements Engineering	7
3	Requirements Analysis and UML	7
4	Data and Information Flow Modeling	7
5	Software Architecture and Design Engineering	7
6	User Interface Design	5
7	Threat Modeling and Security Analysis	10
8	Attack Tree and Security Architecture Refinement	6
9	Product Backlog and Jira/Scrum	7
10	Sprint Execution and Scrum Metrics	7
11	Secure Development and Build Environment	6
12	Secure Coding and Refactoring	4
13	Containerized Development: Docker and Kubernetes	7
14	CI/CD and Security Testing	7
15	Logging, Monitoring, Hardening and Secure Deployment	5
16	Final Security Review	1
	TOTAL	**100**

Assessment Note: Marks should consider both the quality of each artifact and consistency/traceability across phases. Outputs not supported by preceding activities should not receive full credit.

Minimum Final Submission Checklist

☐ SRS and security requirements

☐ Use Case Diagram + two specifications + analysis model

☐ ER Diagram + DFD with trust boundaries

☐ Architecture/component design + UI screens

☐ Asset/CIA + STRIDE + Information Flow + Vulnerability Analysis

☐ Attack Tree

☐ Product Backlog + Jira + two Sprint Plans

☐ Sprint Board + Daily Scrum + Burndown + Velocity + Defects + Review/Retrospective

☐ Source code + refactoring/security fixes + secure-build evidence

☐ Dockerfile + container + Kubernetes deployment evidence

☐ CI/CD pipeline + unit/integration/system tests + fuzzing evidence

☐ Logging/monitoring + hardening/deployment checklist

☐ Final traceability and security review

General Case-Study Instruction

The problem statement/system to be used for the examination will be provided separately by the examiner or assigned to the student/team. Students must apply every phase to that same system. No single domain (examination, banking, hospital, e-commerce, etc.) is prescribed in this question paper.